Tool Calling Under the Hood

What the model reads, and what it writes, when it calls a tool · video 2:18

This lesson answers one question: since a model only reads and writes text, what is a tool call, really? The **chat template** turns tool definitions into instructions the model reads; the model answers with JSON between `<tool_call>` tags; the server or harness **parses** it into a structured call; and the result goes back as more text. Real numbers from Qwen3 4B and Codex CLI show what each step costs, and what happens when the JSON is broken.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 7).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. What the model reads	▶ 0:08–0:33	2	☐
2. What the model writes, and how it is parsed	▶ 0:33–1:05	3	☐
3. How the result goes back	▶ 1:05–1:17	1	☐
4. Tool calls in Codex	▶ 1:17–2:02	3	☐

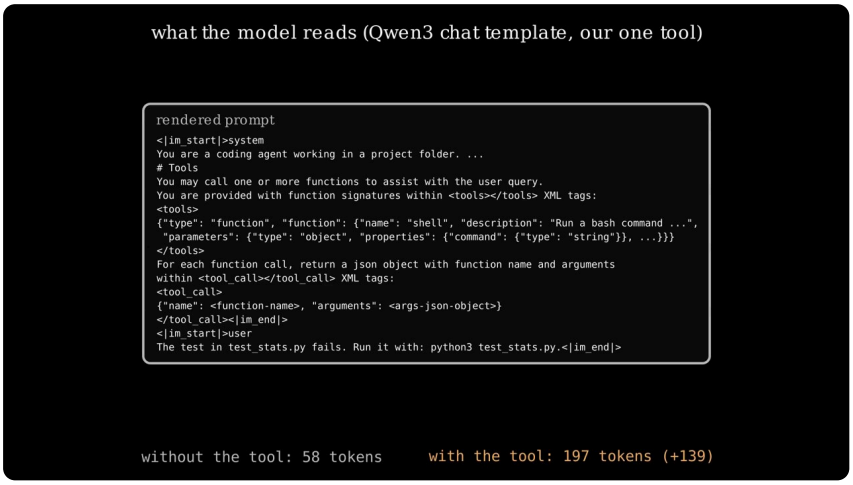
Before you start: episodes 1–2. Code: `code/a03_tool_calling` (`show_prompt.py` needs `transformers` for the tokenizer only; `raw_samples.py` needs a local Ollama with `qwen3:4b`).

CONCEPT 1 OF 4

What the model reads

▶ Video 0:08–0:33

A model has no special channel for tools. The **chat template** (the recipe that turns a list of messages into one long text) writes the tool definitions into the system message, inside `<tools>` tags, followed by an instruction: *to call a function, return a JSON object with its name and arguments within `<tool_call></tool_call>` tags*.



▶ 0:32 Qwen3's chat template turns our tool into instructions in the system message; the tool adds 139 tokens (58 → 197).

For our one small `shell` tool, the first prompt is 197 tokens instead of 58: the tool definition and its instructions cost 139 tokens, on every call. Switching “thinking” off in Qwen3 is also just text: the template pre-fills an empty `<think></think>` block.

KEY IDEA

Tools reach the model as text: definitions and instructions written into the prompt by the chat template.

Check 1: What the model reads

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

With the tool the first prompt is 197 tokens, without it 58. How many tokens does the tool add?

1.2 TRUE OR FALSE

“The model receives tool definitions through a channel separate from its text.”

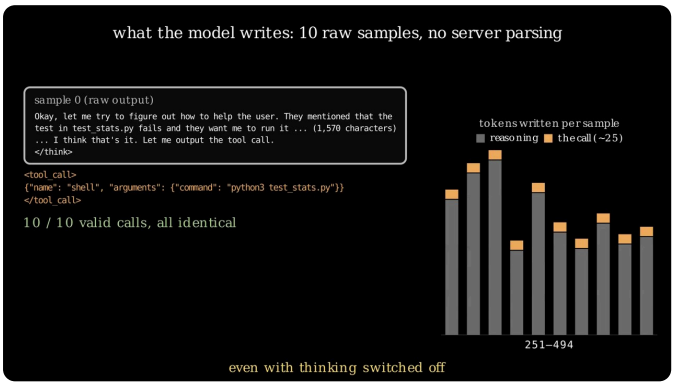
True ☐ False ☐

CONCEPT 2 OF 4

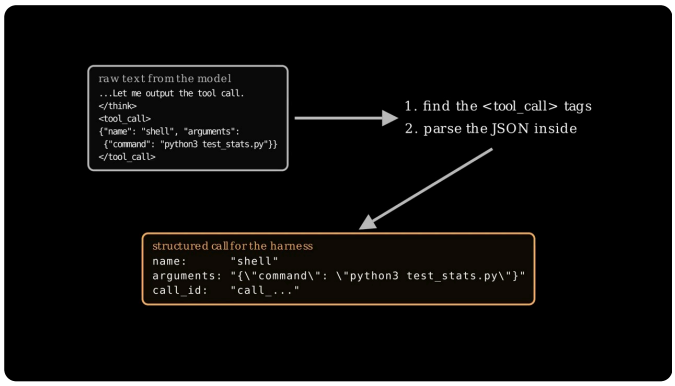
What the model writes, and how it is parsed

▶ Video 0:33–1:05

Sampled 10 times with no server parsing in between (Ollama raw mode, temperature 0.7), Qwen3 4B wrote the same valid call every time: `{"name": "shell", "arguments": {"command": "python3 test_stats.py"}}`, about 25 tokens. Before it, each sample wrote 251–494 tokens in total, nearly all of it its own reasoning, even with thinking “off”.



▶ 0:51 10 raw samples: every one wrote the same valid call (about 25 tokens), after 250–500 tokens of reasoning.



▶ 1:04 Parsing: find the tool_call tags, parse the JSON, hand the harness a name, arguments and a call ID.

Something must find the call in that text: the model server (Ollama does this for its OpenAI-style APIs) or the harness. It searches for the tags, parses the JSON and returns a **structured call**: the tool's name, its arguments (as a JSON string) and a **call ID** that will tie the result to this call.

> **KEY IDEA**
> A tool call is text in an agreed format; parsing turns it into a structured call with a name, arguments and an ID.

Check 2: What the model writes, and how it is parsed

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

In the 10 samples, how many produced a valid tool call?

2.2 PUT IN ORDER

Put the parsing steps in order.

2.3 SHORT ANSWER

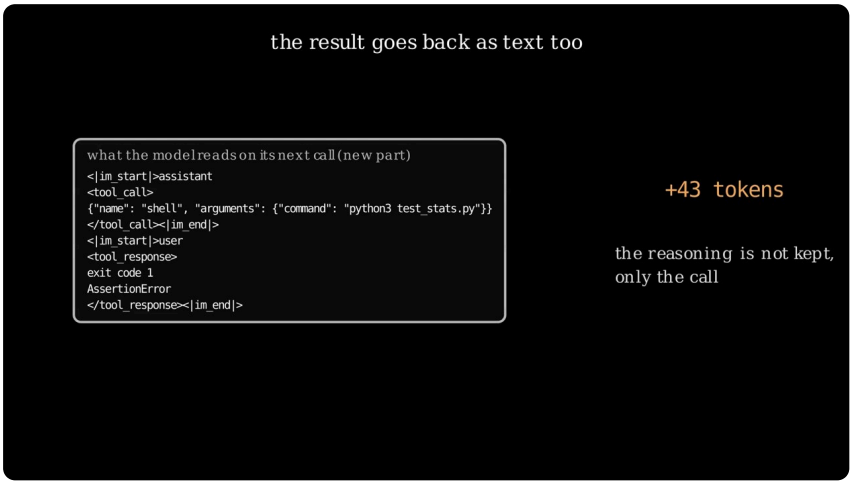
Each sample wrote about 25 tokens of call and up to about 470 tokens of reasoning. Why does that matter for an agent?

CONCEPT 3 OF 4

How the result goes back

▶ Video 1:05–1:17

On the next call, the template writes the model's earlier turn as just its `<tool_call>` block, and the result as a new turn wrapped in `<tool_response>` tags. The call and its result add 43 tokens to the conversation (the call's JSON alone is 19). Qwen3's template does not keep the reasoning that came before the call.



▶ 1:16 The call and the result, as the model reads them on its next call: 43 tokens; the reasoning is not kept.

KEY IDEA

To the model, a tool result is just more text to read, marked with tags so it knows where it came from.

Check 3: How the result goes back

Answer all 1 questions before looking at the answers. Pass mark: all correct.

3.1 TRUE OR FALSE

“In Qwen3's template, the model's reasoning before a tool call is kept in the conversation.”

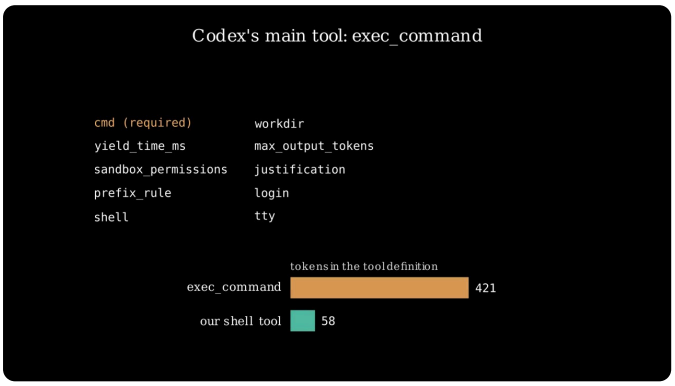
True ☐ False ☐

CONCEPT 4 OF 4

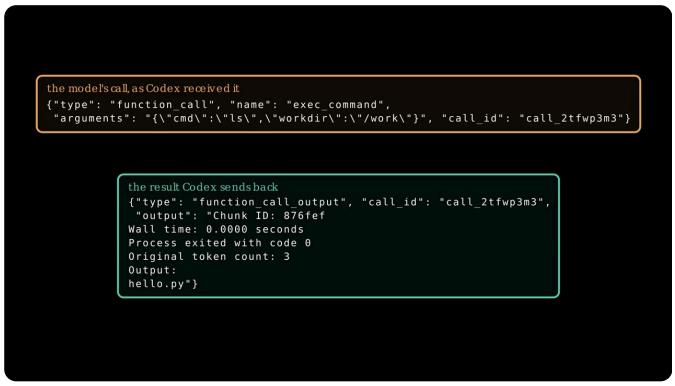
Tool calls in Codex

▶ Video 1:17–2:02

Codex's main tool, `exec_command`, has 10 parameters; only `cmd` is required. The others set the working directory, how long to wait for output (`yield_time_ms`, default 10 s), an output budget (`max_output_tokens`, default 10,000), a request to run outside the sandbox with a justification, a reusable approval prefix, and shell options. Its definition is 421 tokens; our `shell` tool's is 58.

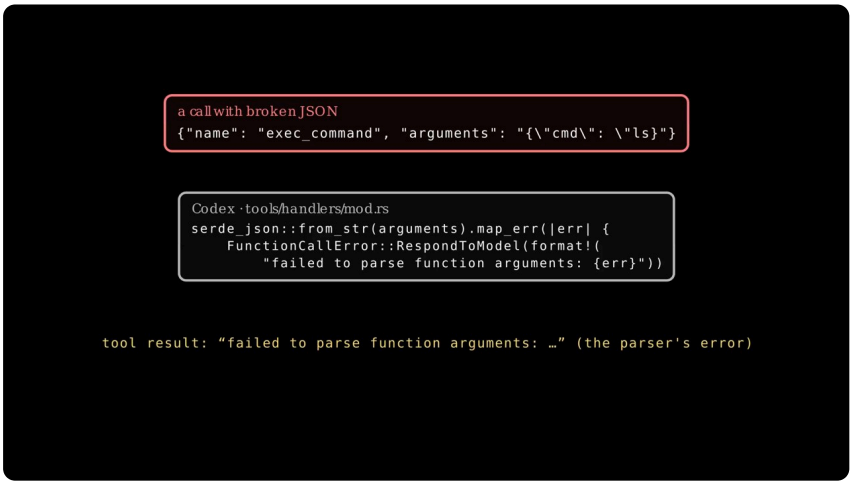


▶ 1:38 exec_command: 10 parameters, only cmd required; 421 tokens of definition against 58 for our shell tool.



▶ 1:50 A real call and result in Codex, matched by call_id; the result starts with a header.

In the captured run the model called `exec_command` with `{"cmd":"ls","workdir":"/work"}` and call ID `call_2tfwp3m3`. Codex's result carried the same ID and started with a header (chunk ID, wall time, exit code, original token count) before the output. If arguments are not valid JSON, Codex (`tools/handlers/mod.rs`) returns `failed to parse function arguments: …` to the model as the result, so it can try again.



▶ 2:01 Broken JSON: Codex sends “failed to parse function arguments” back to the model as the result.

KEY IDEA

Real harnesses use richer tool definitions, label results with the call ID and metadata, and turn errors into results the model can read.

Check 4: Tool calls in Codex

Answer all 3 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

exec_command's definition is 421 tokens, our shell tool's 58. How many times larger?

4.2 MULTIPLE CHOICE

Why does each result carry the call's ID?

A. To encrypt the output

B. So the model can match each result to its call

C. To count tokens

D. Because JSON requires it

4.3 HANDS-ON CODE

Try it yourself. Parse a tool call out of a model's text, returning an error the model can read when it fails.

```
def parse_tool_call(text):
    m = re.search(r"<tool_call>(.*?)</tool_call>", text, re.S)
    if not m:
        return None, "no tool call"
    try:
        return json.loads(m.group(1)), None
    except json.JSONDecodeError as e:
        return None, f"failed to parse function arguments: {e}"
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. What the model reads · rewatch ▶ 0:08–0:33

1.1 **139.** 197 − 58 = 139.

1.2 **False.** The chat template writes them into the system message as text.

2. What the model writes, and how it is parsed · rewatch ▶ 0:33–1:05

2.1 **10.** All 10 parsed, with identical calls.

2.2 **find the tool_call tags → parse the JSON inside → hand the harness a structured call.** Without the tags there is nothing to parse; without parsing there is no call to run.

2.3 **Most of the generated tokens, and most of the time, go to text the harness never uses.** Generation time is proportional to tokens written.

3. How the result goes back · rewatch ▶ 1:05–1:17

3.1 **False.** Only the tool call is kept; the reasoning is dropped.

4. Tool calls in Codex · rewatch ▶ 1:17–2:02

4.1 **About 7.3.** 421 / 58 ≈ 7.26.

4.2 **B.** With several calls in flight, IDs tell which result answers which call.

4.3 **raw_samples.py uses this function on every sample.** Returning the error, instead of crashing, gives the model a chance to fix its call.